

	Hope Foundation's Finolex Academy of Management and Technology, Ratnagiri		
	Department of Computer Science and Engineering (AIML)		
Subject name: Operating System Lab		Subject Code: 2164115	
Class	SE CSE	Semester –IV (NEP 2020)	Academic year: 2025-26
Name of Student	SHAHA IFFAT ANEES		
Roll No	68	Experiment No.	11
Title: Demonstration of functions of operating system for simulation using AI.			

1. Lab objectives applicable: LOB1: To gain practical experience with designing and implementing concepts of operating systems such as system calls, CPU scheduling, process management, memory management, file systems and deadlock handling using C language in Linux environment.			
2. Lab outcomes applicable: LO6: Demonstrate file management, I/O management, and network-based resource sharing in Linux systems and analyze performance.			
3. Learning Objectives: To understand the structure and working of file systems and file operations such as create, open, read, write, and delete and to study I/O management concepts including buffering, spooling, device drivers, and interrupt handling.			
4. Practical applications of the assignment/experiment: File management concepts are used in operating systems and database systems for structured data storage and retrieval.			
5. Prerequisites: Data Structures Computer Architecture			
6. Minimum Hardware Requirements: 1. P-IV PC with 1GB RAM, 320GB HDD, NIC			
7. Software Requirements: 1. Windows or Linux operating systems			
8. Active Link https://colab.research.google.com/drive/1tvz7XpF63gzkw9MBDKvB6gRF-eMBEJTB?usp=sharing			
9. Experiment/Assignment Evaluation:			
Sr. No.	Parameters	Marks obtained	Out of
1	Technical Understanding : Quiz		6
2	Lab Performance		2
3	Punctuality		2
Date of performance (DOP)		Total marks obtained	10

Mrs. G.N.Sawant
Signature of Faculty

CASE STUDY: AI-Based Intelligent File Management System using IFPE

1. Introduction

File management is a fundamental function of an operating system that deals with storing, organizing, and retrieving files efficiently. Traditional systems rely on manual organization and static structures such as folders and directories.

However, modern computing environments demand intelligent systems that can automatically prioritize and organize files based on user behavior.

This case study presents an AI-based Intelligent File Management System using an Intelligent File Prioritization Engine (IFPE), which enhances file handling by predicting file importance

2. Objective

The main objectives of this system are:

- To simulate an OS-based file management system
- To integrate AI for intelligent file prioritization
- To reduce file access time
- To automate file organization into meaningful categories

3. Proposed Solution

The proposed system introduces an AI-based decision engine (IFPE) that:

- Calculates a score for each file
- Categorizes files into:
 - Quick Access (important files)
 - Archive (less important files)
- Provides recommendations for frequently used files
- Enables fast file searching

4. Methodology

The system works in the following steps:

Step 1: Input Collection

The user provides:

- File name
- File type
- Access count (frequency of usage)
- Days since last use

Step 2: Data Storage

- Files are stored in memory
- Data is saved in a JSON file (files_data.json)

Step 3: AI-Based Scoring Mechanism

The system calculates a score using:

$\text{Score} = (\text{Access Count} \times 2) - \text{Days Since Last Use} + \text{File Type Priority}$

File Type Priority:

- Documents (pdf, doc) $\rightarrow +3$
- Images (jpg) $\rightarrow +1$
- Others $\rightarrow 0$

Step 4: File Classification

Based on score:

Score	Category
> 5	Quick Access
≤ 5	Archive

Step 5: Recommendation System

- Files are sorted based on score
- Top 3 files are recommended

Step 6: Visualization

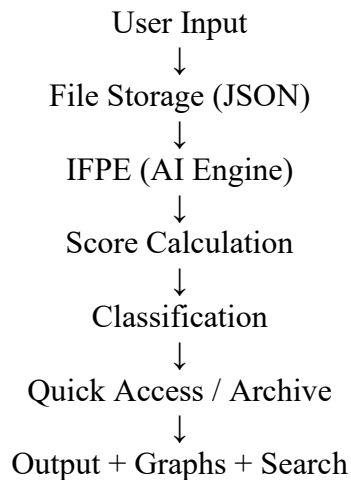
The system generates:

- Bar graph (file distribution)
- Pie chart (percentage distribution)

Step 7: Search Function

- Users can search files by name
- System retrieves files instantly

Step 8: System Architecture



5. Implementation

The system was implemented using:

- Python (core logic)
- Tkinter (GUI interface)
- Matplotlib (graphs)
- JSON (data storage)

Source code:

```
import tkinter as tk
from tkinter import messagebox
import matplotlib.pyplot as plt
import json
import os

# File storage
files = []

DATA_FILE = "files_data.json"

# Load saved data
def load_data():
    global files
    if os.path.exists(DATA_FILE):
        with open(DATA_FILE, "r") as f:
            files = json.load(f)
            for file in files:
                listbox.insert(tk.END, f'{file["name"]} ({file["type"]}) | Access: {file["access"]} | Last used: {file["days"]} days')

# Save data
def save_data():
    with open(DATA_FILE, "w") as f:
        json.dump(files, f)

# Add file
def add_file():
    name = entry_name.get()
    ftype = entry_type.get()

    try:
        access = int(entry_access.get())
        days = int(entry_days.get())
    except:
        messagebox.showerror("Error", "Enter valid numbers")
        return

    # Duplicate detection
    for file in files:
        if file["name"] == name:
            messagebox.showwarning("Duplicate", "File already exists!")
            return

    files.append({"name": name, "type": ftype, "access": access, "days": days})
    listbox.insert(tk.END, f'{name} ({ftype}) | Access: {access} | Last used: {days} days')
```

```

save_data()

entry_name.delete(0, tk.END)
entry_type.delete(0, tk.END)
entry_access.delete(0, tk.END)
entry_days.delete(0, tk.END)

# IFPE AI Engine
def process_files():
    quick = []
    archive = []
    scores = []

    for file in files:
        # Score calculation
        score = (file["access"] * 2) - file["days"]

        # File type priority
        if file["type"] in ["pdf", "doc"]:
            score += 3
        elif file["type"] in ["jpg"]:
            score += 1

        scores.append((file["name"], score))

    if score > 5:
        quick.append((file["name"], score))
    else:
        archive.append((file["name"], score))

# Display results
result_text.delete(1.0, tk.END)
result_text.insert(tk.END, "=== Intelligent File Prioritization Engine (IFPE) ===\n\n")

result_text.insert(tk.END, "Quick Access:\n")
for f, s in quick:
    result_text.insert(tk.END, f"• {f} (Score: {s})\n")

result_text.insert(tk.END, "\nArchive:\n")
for f, s in archive:
    result_text.insert(tk.END, f"• {f} (Score: {s})\n")

# Recommendations (Top 3)
scores.sort(key=lambda x: x[1], reverse=True)
result_text.insert(tk.END, "\nTop Recommended Files:\n")
for name, score in scores[:3]:
    result_text.insert(tk.END, f"• {name} (Score: {score})\n")

# Graphs

```

```

labels = ["Quick Access", "Archive"]
values = [len(quick), len(archive)]

plt.figure()
plt.bar(labels, values)
plt.title("File Distribution (IFPE)")
plt.xlabel("Category")
plt.ylabel("Number of Files")
plt.show()

plt.figure()
plt.pie(values, labels=labels, autopct='%1.1f%%')
plt.title("File Distribution Percentage")
plt.show()

# Search function
def search_file():
    keyword = entry_search.get().lower()
    result_text.delete(1.0, tk.END)

    found = False
    for file in files:
        if keyword in file["name"].lower():
            result_text.insert(tk.END, f"Found: {file['name']} ({file['type']})\n")
            found = True

    if not found:
        result_text.insert(tk.END, "No file found")

# GUI
root = tk.Tk()
root.title("AI File Management System - IFPE")
root.geometry("600x650")

tk.Label(root, text="File Name").pack()
entry_name = tk.Entry(root)
entry_name.pack()

tk.Label(root, text="File Type (pdf, doc, jpg, mp4)").pack()
entry_type = tk.Entry(root)
entry_type.pack()

tk.Label(root, text="Access Count").pack()
entry_access = tk.Entry(root)
entry_access.pack()

tk.Label(root, text="Days Since Last Use").pack()
entry_days = tk.Entry(root)
entry_days.pack()

```

```

tk.Button(root, text="Add File", command=add_file).pack(pady=5)

listbox = tk.Listbox(root)
listbox.pack(fill=tk.BOTH, expand=True)

tk.Button(root, text="Run AI (IFPE Engine)", command=process_files).pack(pady=10)

# Search
tk.Label(root, text="Search File").pack()
entry_search = tk.Entry(root)
entry_search.pack()

tk.Button(root, text="Search", command=search_file).pack(pady=5)

result_text = tk.Text(root, height=12)
result_text.pack()

# Load previous data
load_data()

root.mainloop()

```

6. Result and Analysis

AI File Management System - IFPE

File Name

File Type (pdf, doc, jpg, mp4)

Access Count

Days Since Last Use

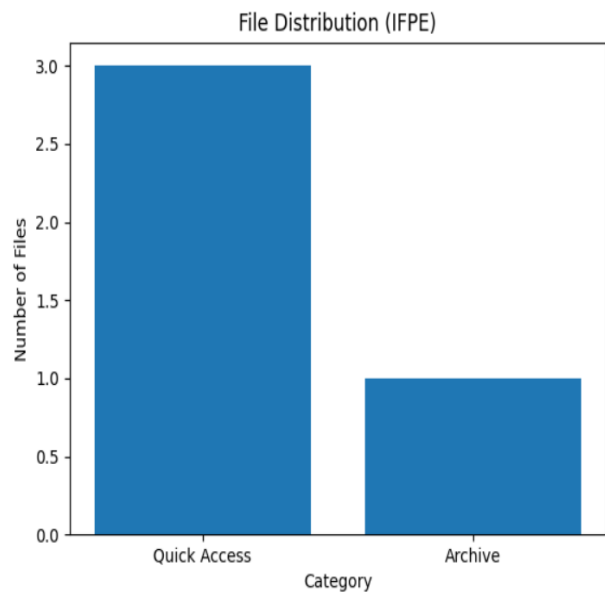
Add File

iffat (pdf) | Access: 6 | Last used: 0 days
os (doc) | Access: 3 | Last used: 2 days
functions of os (jpg) | Access: 10 | Last used: 4 days
old file (jpg) | Access: 1 | Last used: 12 days

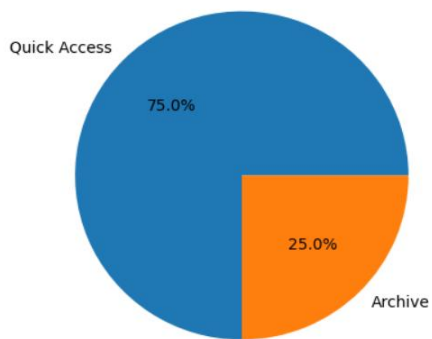
Run AI (IFPE Engine)

Search File

Search



File Distribution Percentage



Search File

os

Search

Found: os (doc)
Found: functions of os (jpg)

```
=== Intelligent File Prioritization Engine (IFPE) ===  
  
Quick Access:  
• iffat (Score: 15)  
• os (Score: 7)  
• functions of os (Score: 17)  
  
Archive:  
• old file (Score: -9)
```

Archive:
• old file (Score: -9)

Top Recommended Files:
• functions of os (Score: 17)
• iffat (Score: 15)
• os (Score: 7)

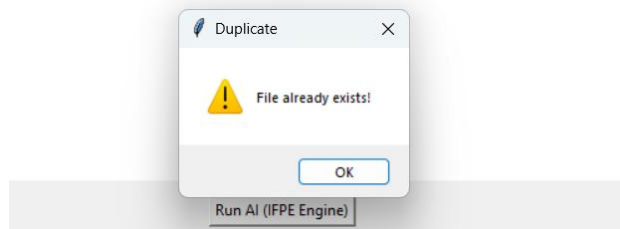
File Name
iffat

File Type (pdf, doc, jpg, mp4)
pdf

Access Count
6

Days Since Last Use
0

Add File



7.Observations:

- Frequently used files are prioritized
- Old and unused files are archived
- System improves file accessibility

8. Advantages

- Reduces manual effort
- Improves file retrieval speed
- Intelligent decision-making
- User-friendly interface
- Scalable for real-world systems

9. Limitations

- Uses rule-based AI (not fully learned model)
- Limited file attributes
- Does not integrate with real OS

10. Conclusion

The AI-based file management system successfully demonstrates how artificial intelligence can enhance traditional operating system functions. By using a scoring mechanism, the system intelligently prioritizes files, improves accessibility, and reduces user effort.

This approach highlights the potential of AI in optimizing core OS functionalities.

Learning Outcomes Achieved: Analyse file management functions of Operating System by making simulation using AI.

Conclusion:

1. Applications of the Studied Technique in Industry:

Used in OS, databases, and cloud storage for efficient and reliable file handling.

2. Engineering Relevance:

Important for system programmers, storage architects, and software engineers in file systems and real-time applications.

3. Skills Developed:

Enhances system-level programming, problem-solving, debugging, and performance analysis.

References:

- [1] Achyut Godbole and Atul Kahate, *Operating Systems*, McGraw Hill Education, 3rd Edition
- [2] Andrew S. Tanenbaum, *Operating System Design and Implementation*, Pearson, 3rd Edition
- [3] Maurice J. Bach, *Design of UNIX Operating System*, PHI
- [4] Sumitabha Das, *UNIX: Concepts and Applications*, McGraw Hill, 4th Edition
- [5] William Stallings, *Operating System: Internals and Design Principles*, Prentice Hall, 8th Edition
- [6] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne, *Operating System Concepts*, John Wiley & Sons, 9th Edition

